P2.11 — a keyed build in under 90 seconds

Status: draft, 10 October 2026. Not started. The target's meaning was decided the same day: option 2 (see "The decision before work starts"). **Position:** after P2.10, before Phase 3

Why this exists as its own item

A keyed build of a 442-person export takes about five minutes, behind a spinner (D-72). On the deployed app it has taken about 320 s. That is a stranger's first impression, on a shared container.

This item makes the build faster without changing what the classifier says. Everything below rests on measurements taken on 10 October, after the stage timings landed (`0fe5ca2`). Earlier figures for this item, including a 60 s target, came from arithmetic before anything was measured. They are withdrawn, not carried.

P2.11 changes how batches are sent, not what is sent. The prompt, the model, the batch size and the composition of each batch stay as they are. Config E is not touched.

What was measured

Two keyed builds through the app

Run on 10 October on the local machine. The app was at HEAD (`0fe5ca2`) with `NV_TIMING=1`, and classification ran on the project key. The file uploaded was the synthetic export, `data/synthetic/Connections.csv`, identified by its parse lines in the log: 442 rows, header at line 3, 12 missing a company. It has 442 people and 384 distinct titles. Both runs used a fresh session, so the session's classification cache was empty and all 384 titles went to the API in 20 batches: 19 of 20 titles and one of 4.

- **Cold:** the first build after the server started. The heavy libraries were imported and UMAP compiled during it.
- **Warm:** a second upload in a new tab, on the same server process.

The raw log is `docs/harness_runs/p2_11_timing_20261010.log`. The two network address lines of Streamlit's startup banner were removed before it was committed. Nothing else in the file was changed.

Stage	Cold	Warm
parse	0.02 s	0.02 s
embed	6.18 s	1.73 s
umap	15.99 s	0.77 s
graph	0.15 s	0.16 s
classify	306.80 s	295.21 s
metrics	1.19 s	1.26 s
other	1.01 s	0.01 s
total	**331.34 s**	**299.16 s**
full batches (19): mean	15.93 s	15.37 s
full batches: standard deviation	3.12 s	2.13 s
full batches: range	10.53 to 21.51 s	12.71 to 20.51 s
output tokens, all batches	33,173	32,397
canvas render, server side	0.10 s	0.03 s

The summary lines in the log give `mean_batch` as 15.32 s and 14.75 s. Those include the 4-title batch. The means in the table are over the 19 full batches.

Classification was 93% of the cold build and 99% of the warm one. The cold build cost 32 s more, almost all of it in embed and UMAP warming up.

Three keyless builds in one process

Run the same day, in one Python process, with `classify=False`, on `data/raw/Connections.csv`, the real export (452 rows, 442 people after parsing). This is a different file from the synthetic upload above, with the same people count.

	Build 1	Build 2	Build 3
embed	5.75 s	1.76 s	1.73 s
umap	14.53 s	0.74 s	0.72 s
total	22.05 s	3.39 s	3.30 s

The cost outside classification is paid once per process, not once per upload. UMAP isn't slow: its first fit in a process compiles, and later fits take under a second. That matches the P2.9b spike (10.4 s for the first fit, 0.6 s for the second). A first build on a cold disk cache took 35.8 s earlier the same day.

Constructing the encoder alone, after those builds, took 1.32 s. The app builds it again on every upload (D-88).

The two deployed figures: 320 s and 230 s

The deployed upload's cold or warm state isn't known. Each deploy ends with a reboot, so it may well have been the first upload after a restart. On Cloud a cold start costs more than it does locally. The spike measured 36.0 s to import torch, sentence-transformers and umap, 3.8 s to load the model, and 10.4 s for the first UMAP fit. **The defensible statement is that classification was at least about 270 s of the deployed 320 s.** Nothing measured supports a stronger one.

A second deployed figure, 10 October: about 230 s. A stopwatch estimate, from the live check on the deployed cut (`21e5500`, in the public repo), which has no stage timings. The deploy log settles two things about it: - **The file was the synthetic export,** the same file as the local runs above, identified by the same parse lines. - **The container was warm.** Its first build since the 9 October provision was at 18:29 UTC, and nothing restarted it before the live check's builds at 22:08 to 22:09 UTC.

What the stopwatch spanned is not settled. The log shows three parses in those two minutes, and why is open (D-90). Within one session, runs don't overlap, so any rebuild adds seconds after a build rather than running beside it. By the same reasoning as above, classification was at least about 180 s of the 230 s.

The 90 s between the two deployed figures is unexplained. Locally, the two runs differed by 32 s, all of it cold start. The candidates, none checked: - a cold container for the 320 s against the warm one for the 230 s, worth about 50 s by the spike's figures (the 320 s run's state isn't known) - a different export or number of uncached titles for the 320 s (the 230 s used the synthetic export, with all 384 titles uncached in a fresh session) - variation in output tokens, which sets each batch's time

Both figures stand. The target is absolute, so it doesn't move with either. The distance to it does: from 230 s the build needs to get about 2.6 times faster, and from 320 s about 3.6 times.

Batch time is output tokens

Across the 20 batches of each run, a batch's duration tracks its output tokens almost exactly.

	r (19 full batches)	Rate
Cold run	0.981	about 89 tokens a second
Warm run	0.949	about 93 tokens a second

Output tokens include Sonnet 5's adaptive thinking. A batch is slow because it generated more, mostly by thinking longer, not because the network or the service was slow. This is a volume problem that looks like a latency problem. It also explains why concurrency gains less than the mean batch time suggests: a group of concurrent batches finishes when its slowest batch does, and the slowest is the one that thought the longest.

The two levers

Lever 1: send batches concurrently

This changes wall time and nothing else. Each batch is the same request it is today. Only when it is sent changes.

Lever 2: fewer output tokens (blocked)

Less thinking, or a shorter `why` field, would cut time and cost in proportion. It is not available inside P2.11, for three reasons:

- **It is a classifier change.** What the model generates changes, so what it classifies may change. Re-tuning config E is ruled out (`docs/phase2_scope_revision.md`): it sits at the human ceiling, 63.5% precision on the blind 100.
- **`MAX_TOKENS` is a ceiling, not a target.** It is 8,000 because D-42 proved that a lower ceiling starves the text block: at 2,000 a batch spent the lot on thinking and the JSON never arrived. Lowering it truncates, it does not shorten.
- **"Unchanged" can't currently be shown.** Reopening config E would need D-47's remeasurement repeated (`tools/remeasure_config_e.py`): all 100 blind titles, precision and coverage unchanged, zero fabrications. But the 63.5% is one draw from a non-deterministic classifier, and its spread was never measured (D-69). Until D-69's three re-runs put an error bar on it, there is no way to tell a real change from noise. That work comes first.

Lever 2 is its own item, if it is ever taken up. **Concurrency is the only lever available without touching the locked configuration.**

The concurrency projection

This is a projection, not a measurement. It replays the 20 measured batch times of each run through a pool of N workers, where a worker takes the next batch as soon as it finishes one. That is how a thread pool would send them.

The assumption: **each batch takes as long under concurrency as it did alone.** That is untested. Rate limiting, or the SDK retrying a refused request, would break it.

The cost outside classification is taken as measured: 24.5 s for a local cold build and 4.0 s for a warm one. The Cloud cold figure, 52.1 s, is an estimate. It adds the spike's Cloud measurements (imports 36.0 s, model load 3.8 s, embed 0.5 s, first UMAP fit 10.4 s, from `docs/p2_9b_spike_record.md`, 22 September) to this run's warm graph and metrics times.

Workers	classify, cold run / warm run	Local cold total	Warm total	Cloud cold total (estimate)
1 (today)	306.8 / 295.2 s	331 s	299 s	359 s
4	80.8 / 77.5 s	105 s	82 s	133 s
5	64.6 / 64.3 s	89 s	68 s	117 s
6	54.7 / 55.8 s	79 s	60 s	107 s
8	46.2 / 43.6 s	71 s	48 s	98 s
10	34.3 / 33.3 s	59 s	37 s	86 s
12	33.3 / 31.4 s	58 s	35 s	86 s

The Cloud column doesn't fit the 230 s figure. It assumes Cloud batches take as long as the local ones, and at one worker it projects 359 s. The deployed app has measured 320 s and about 230 s. If the 230 s build sent every title uncached, Cloud batches ran faster than local ones, or the build sent less. Either way the Cloud column is pessimistic, and fewer workers may meet the target on Cloud than it shows. Faster Cloud batches would not point at the local machine. A batch's time is spent waiting on the API: its duration tracks output tokens at about 90 a second, which is the model's generation speed on Anthropic's side, and the fixed cost per call is small. The likelier causes are fewer output tokens in that run, a faster service at that hour, or fewer batches sent. Only an instrumented Cloud run can say which, and P2.11.4 is that run. Until then, the column is an upper estimate.

Past about 10 workers, the slowest single batch sets the floor: 21.5 s in the cold run and 20.5 s in the warm one. More workers stop helping.

An earlier projection, given in conversation and never written into any record, grouped batches into fixed waves of N. That made 8 workers look like the minimum. A pool does better, which is why this table needs fewer workers for the same total.

The target

Under 90 seconds for a cold build of the 442-person export, on a fresh session, measured with `NV_TIMING=1` and the log committed.

Warm is the common case on a deployed app: a container serves visitors one after another until it restarts. Cold is the worst case a visitor meets. The target is set on the worst case.

The decision before work starts

Decided, 10 October 2026: option 2, the deployed cold build, met by concurrency plus a warm-up. The options are left below as they were weighed.

The reason is not the better numbers. Cold overhead is dead time a visitor pays for nothing. Importing the libraries and building the encoder while someone reads the landing page is work that was going to happen anyway, moved to a moment when nobody is waiting. That is an improvement whether or not it saves a worker. Fewer workers for the same target is the second-order gain: more headroom against a rate limit nobody has checked yet. The warm-up makes the rate-limit question less likely to bite. It doesn't remove it.

Which cold? The projection meets 90 s with 6 workers locally (79 s), but needs about 10 on Cloud (86 s, estimated). The Cloud cold start is about twice the local one. There are three ways to settle it:

1. **The target is the deployed cold build, met by concurrency alone.** This needs about 10 workers, roughly 65,000 output tokens a minute (see "Rate limits"), with the least margin. 86 s against a 90 s target, on an estimate, is not a comfortable pass.

2. **The target is the deployed cold build, met by concurrency plus a warm-up.** Import the heavy libraries, build the encoder and run one small UMAP fit in the background once the landing page has rendered. A visitor's upload then starts warm, unless it arrives within the first minute after a restart. 6 to 8 workers then meet 90 s with room to spare, and the rate-limit question is easier. The cost is about 1.1 GB held from startup, which the first build would hold anyway. The landing page also shares the CPU with the imports for about a minute. This goes naturally with D-88: the warm-up builds the encoder once and keeps it.
3. **The target is the local cold build.** That is met by 6 workers, but it is not what a visitor experiences. Not recommended.

The second is the one that meets the target on what a visitor sees without leaning on an estimate at the edge. The choice is the owner's. If neither 1 nor 2 is available (for example, the rate limit refuses even 6 workers), the honest outcome is that P2.11 can't reach 90 s without a classifier change. That goes back as a decision, rather than 115 s being accepted quietly.

The work

In this order. Each item is a prerequisite for the one after it.

P2.11.1 — The encoder built once per process (D-88)

`_load_model()` keeps its model for the life of the process. That saves about 1.3 s on every upload, warm or cold, and makes the spike record's sentence true. D-88's dated note in the spike record says from which commit.

It lands first, on its own, because the warm-up depends on it. Caching alone doesn't stop a race: two threads that ask an empty cache at the same moment both see it empty, and both build the encoder. `functools.lru_cache` doesn't lock around the call either. **The cache is guarded by a lock.** A second caller waits for the first build rather than starting its own. A failed build caches nothing, so the next caller tries again.

Done when: two builds in one process construct the encoder once, and a test with two threads calling `_load_model()` at the same moment, against a slow fake constructor, sees one construction.

P2.11.2 — Warm-up once the landing page has rendered

A background thread, started once per process, that imports the heavy libraries, builds the encoder through P2.11.1's cache, and runs one small UMAP fit to trigger the compile. Requirements:

- **It doesn't block the landing page.** The thread starts after the page's elements have been sent, never before. A warm-up that delays the first render has made the visitor's experience worse to improve a number. Measured: the landing page's time to first render, with and without the warm-up, at three runs each.
- **A build that arrives during the warm-up doesn't race it.** It waits for the one encoder through P2.11.1's lock rather than building a second. The heavy imports are already serialised by Python's import lock.
- **It degrades silently, for the visitor.** Any exception in the thread is caught, nothing is cached on failure, and the build that follows loads what it needs itself, as it does today. A failed warm-up means a slower build, never a failed one. It is silent on the page, not in the log: the thread prints one line to stdout, success or failure, so a warm-up that never works is visible to whoever reads the deploy log.
- **It compiles what the build will use.** The UMAP fit goes through `run_umap_3d` with `_umap_params`, on float32 input with 384 columns, the encoder's output, and enough rows that `n_neighbors` isn't clamped. numba compiles per type signature, so a fit on different input could compile code the build then doesn't use. P2.11.4's `umap` line for the first upload is the check.

- **It doesn't run in tests.** `tests/test_app_smoke.py` runs `app.py` through AppTest, so without a switch every smoke test would start a thread that imports torch and outlives its test. A thread left running by an earlier AppTest-based test is D-87's leading hypothesis. The switch is an autouse fixture in `tests/conftest.py`, with a session backstop that fails the run if a warm-up thread was started anyway. That follows the redirect pattern the cache and the ledger use.

Done when: a cold server's first upload shows `embed` and `umap` at their warm values, the landing page's first-render time is unchanged within the three-run spread, and a warm-up forced to raise leaves the next build succeeding.

P2.11.3 — Concurrent batches in `classify_claude_outcome`, at 6

Check the account's output-token rate limit in the Anthropic Console before this is built (see "Rate limits"). A ceiling discovered through 429s during a timed run wastes the run.

The batch loop becomes a bounded pool of 6 workers. It is tuned upward only if P2.11.4 misses the target. What it must keep:

- **The batches are the ones sent today.** The same 20-title slices, in the same composition. Batch context can affect answers, so re-slicing would be a classifier change. A test asserts the set of requests is identical to a sequential run's.
- **The caps still hold (D-81).** Today the meter is checked before each batch and charged after it. With N batches in flight, a cap can be overrun by up to N-1 batches that are already paid for. Each dispatch therefore reserves its estimate before it is sent, and releases the reservation when the actual charge lands. The pre-flight check is unchanged.
- **An API error still stops the run.** No new batch is sent after an error. Batches already in flight are paid for, so their answers are kept and cached. The rest get the error's reason, as today.
- **One writer for the cache.** Workers return records. The calling thread merges them into the session's cache and the results, so the cache mapping is never written from two threads.
- **No Streamlit calls from workers.** None are made today, and worker threads have no script context. D-72's progress indicator, when built, updates from the calling thread as batches complete.
- **The timing lines stay true.** The batch lines report each batch's own elapsed time, and the classify stage reports wall time. The summary gains the worker count.

P2.11.4 — Re-measure Cloud cold against the target

Two keyed builds of the same 442-person export through the deployed app, cold (the first upload after a reboot) and warm (a second, in a new tab), with `NV_TIMING = "1"` at the root of the app's secrets. Streamlit copies root-level secrets into the environment. Both logs go under `docs/harness_runs/`. This is also where the rate-limit question is answered in practice: a 429 shows as a slow batch and as an SDK retry in the log.

Each run reports, from its batch lines, two figures: the mean duration over the full 20-title batches, and output tokens per second. The summary's `mean_batch` includes the short final batch, so it isn't the comparable figure. Together the two separate the explanations for a Cloud batch being faster or slower than a local one. The same tokens a second with fewer tokens means the run generated less. More tokens a second means the service was faster at that hour.

For the cold run, the upload is timed to start after the warm-up's log line, and a third run uploads within seconds of the reboot. That records what the visitor who arrives first meets, the case the warm-up can't fully cover. Before that, the app isn't up at all while Cloud installs dependencies: 42 s and 1 min 9 s on the two provisions in the 10 October log. The run records the time from the reboot to the page answering, separately from the build.

Done when: the deployed cold build is under 90 s, and the classifications are unchanged in kind. No item in this document changes a prompt, the model, the batch size or a batch's composition, and the tests say so. If 6 workers miss, the count goes up one step at a time, and each step is measured.

Rate limits (open)

An average batch generated 1,659 output tokens over about 15 s, so one worker generates about 6,500 output tokens a minute.

Workers	Output tokens a minute, roughly
6	39,000
8	52,000
10	65,000

Requests a minute stay low, under 50 at 10 workers. Whether the account's output-token limit for Sonnet 5 allows these rates has not been checked. It depends on the account's usage tier, shown in the Anthropic Console. **Check it before P2.11.3 is built.** If the limit sits below the rate the chosen worker count needs, the projection's assumption fails, and the SDK's retries add the waiting back.

What P2.11 is not

- **Not a classifier change.** No prompt, model, batch size, `MAX_TOKENS` or thinking setting changes. Lever 2 is out of scope (see above).
- **Not D-72.** The progress indicator and what a disconnect does stay in D-72. P2.11 shortens the window D-72 describes and builds on the same stage boundaries, but it doesn't close it.
- **Not a change to the memory ceiling.** The 3,072 MB ceiling and the 1,332 MB measured peak are unchanged. Concurrency holds a few more responses at once and nothing large, and two uploads at once are still two builds. What changes is when memory is held. The warm-up keeps about 1.1 GB resident from startup on every container, including one that only ever serves the demo and would never have loaded torch before.

Risks worth naming

- **The projection's assumption.** Every figure in the concurrency table assumes concurrent batches run at their solo speed. P2.11.4 is the test of it. Nothing before then should quote the table as a result.
- **The Cloud estimate's age.** The Cloud cold overhead comes from a spike on 22 September, with a different harness and before the in-app build existed. Measuring it directly is part of P2.11.4.
- **Spend runs faster.** At 8 workers, a build's spend arrives in under a minute rather than five. The caps hold through the reservations in P2.11.3. If the reservation logic were wrong, though, the overrun would arrive just as fast.
- **Two samples.** The batch statistics are two runs of 20 batches, one export, one machine, one day. They agree closely (means of 15.9 and 15.4 s over the full batches, r of 0.98 and 0.95), but they are two samples.